

PSS101(OR4000)通信协议

协议格式定义

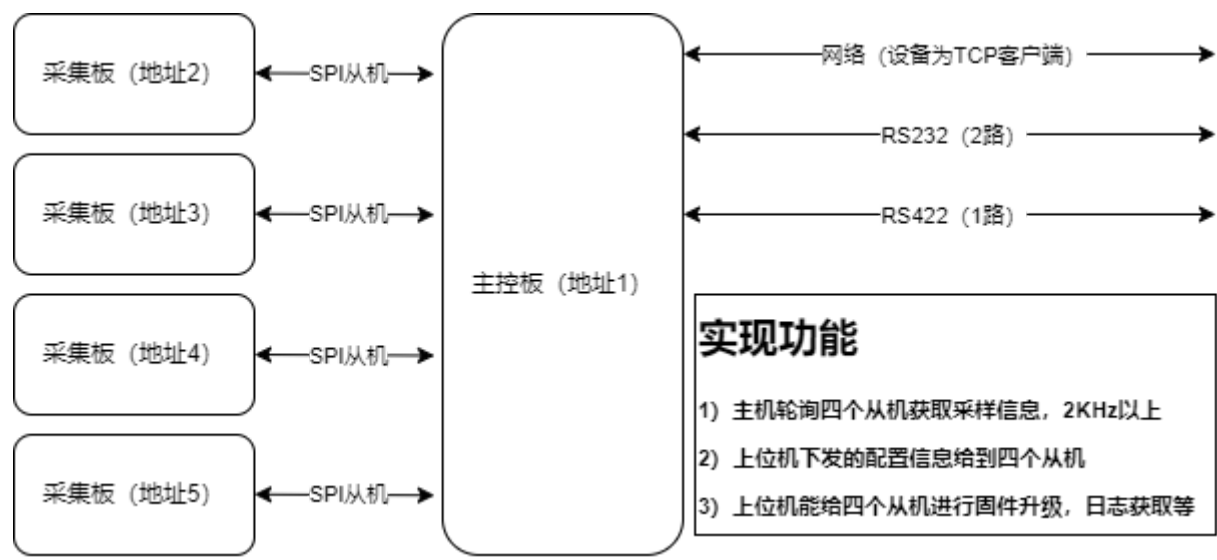
包头						数据		
	起始符	设备地址	命令号	操作类型	数据长度	CRC16	数据	CRC16
数据类型	uint8_t	uint16_t	uint8_t	uint8_t	uint16_t	uint16_t		uint16_t
字节长度	1	2	1	1	2	2	不定长	2
范围	#	0-65535	0-255	0-10	0-65535	0-65535		0-65535

注：

- 此包头并非TCP包头，是传输数据中自定的识别设备地址及命令的数据段。
- 本协议起始符固定为字符'#'，十六进制为0x23。
- 操作类型定义，0为基础读，1为基础写，10为消息确认，2-5为自定义读，6-9为自定义写。
- 数据长度仅为数据部分长度，不包含数据后的CRC。
- 数据区中的所有uint16_t或uint32_t数据，先发送低字节。
- 协议中第一个CRC16是对包头的校验，第二个CRC16是对数据的校验，且CRC16采用业界通用的ModBus RTU CRC算法。
- 协议包头数据结构和协议数据部分1字节对齐。如下示例：

```
/* 包头结构体 */
typedef __packed struct {
    uint8_t  sop;          /* '#' */
    uint16_t dev_id;       /* 设备地址 */
    uint8_t  cmd;          /* 命令 */
    uint8_t  op_type;      /* 操作类型 */
    uint16_t data_len;     /* 载荷数据长度（字节数，不包括CRC16） */
    uint16_t crc;          /* 包头CRC16 */
} prot_tp1_head_t;
```

设备通信链路框图



设备地址定义

- 0：代表广播地址（注意广播时不应答）
- 1：代表主控板地址
- 2、3、4、5： 分别代表4块采集板地址

注意：示例表中黑体的数据为变化项

1 设备基础功能协议

4号命令： 设备软硬件版本查询

设备版本查询

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	4	0	0	无
设备	设备地址	4	0	6	软件版本+硬件版本

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据	数据CRC16
平台	23	01 00	04	00	00 00	09 23		
主控板	23	01 00	04	00	06 00	0A 83	01 00 00 01 00 01	91 CA

版本结构说明：

	数据类型	长度	范围
主要版本号	uint8_t	1	1~255
次要版本号	uint8_t	1	0~255
修正版本号	uint8_t	1	0~255

- 版本号由主要版本号、次要版本号、修正版本号构成，软硬件版本号均符合此规则。
- 软件版本：设备应用程序版本。
- 硬件版本：同型号产品硬件版本。
- 版本策略：依据《和其光电嵌软组版本管理规则》执行。

5号命令：设备重启

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	5	0	0	无
设备	设备地址	5	0	0	无

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据	数据CRC16
平台	23	01 00	05	00	00 00	08 DF		
主控板	23	01 00	05	00	06 00	08 DF		

2 以太网功能协议

20号命令：设备MAC地址上报

时序	地址	命令号	操作类型	数据长度	数据
平台	设备编号	17	10	0	无
设备	设备编号	17	0	6	MAC地址

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据	数据CRC16
平台	23	01 00	11	0A	00 00	2D 2D		
主控板	23	01 00	11	00	06 00	0E 8F	AA BB CC	23 45

数据结构如下：

```
uint8_t device_mac[6]; /* 以太网MAC地址 */
```

3 固件升级协议

50号命令：升级通知（上位机3次重试）

（1）若版本号符合升级要求，设备将升级确认

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	50	1	15	软件版本号+数据包总大小+单包大小+数据包总数+数据包CRC32
设备	设备地址	50	1	15	软件版本号+数据包总大小+单包大小+数据包总数+数据包CRC32

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据(版本号110 总大小65535 单包大小1024 包总数64)	数据CRC16
平台	23	01 00	32	01	0F 00	52 9B	01 01 00 23 8F 02 00 04 66 40	68 AC 00 23 45

源 头	#	地 址	命 令 号	操 作 类 型	数 据 长 度	包 头 CRC16	数据(版本号110 总大小65535 单包大小1024 包总数64)	数据 CRC16
主控 板	23	01 00	32	01	0F 00	52 9B	01 01 00 23 8F 02 00 04 66 40 00 23 45	68 AC

(2) 若版本号不符合升级要求，设备将拒绝升级

时 序	地 址	命 令 号	操 作 类 型	数 据 长 度	数 据
平 台	设备地 址	50	1	15	软件版本号+数据包总大小+单包大小+数据包总 数+数据包CRC32
设 备	设备地 址	50	1	0	无（给文本提示）

示例：

源 头	#	地 址	命 令 号	操 作 类 型	数 据 长 度	包 头 CRC16	数据(版本号110 总大小65535 单包大小1024 包总数64)	数据 CRC16
平台	23	01 00	32	01	0F 00	52 9B	01 01 00 23 8F 02 00 04 66 40 00 23 45	68 AC
主控 板	23	01 00	32	01	00 00	52 9B		

在正式开始升级数据传输前，平台与设备进行升级信息的确认，包含：

- 升级软件版本号(版本号由主要版本号、次要版本号、修正版本号构成)
- 数据包总大小
- 单包数据大小
- 数据包总数
- 数据包 CRC32
- 升级确认
 - 升级软件版本号大于设备软件版本号（断包续传注意必须是同一bin文件）
 - 主控芯片一致（硬件版本号一致）
- 拒绝升级
 - 升级软件版本号小于等于设备软件版本号

- 主控芯片或硬件不一致

数据结构解析：

	数据类型	长度	范围
软件版本号	uint8_t*3	3	
数据包总大小	uint32_t	4	1~262144
单包数据大小	uint16_t	2	1-65535
数据包总数	uint16_t	2	1-65535
数据包 CRC32	uint32_t	4	1~262144

51号命令：升级数据传输

时序	地址	命令号	操作类型	数据长度	数据
设备	设备地址	51	1	7	软件版本号+包序号+数据包总数
平台	设备地址	51	1	7+数据分包	软件版本号+包序号+包大小+数据分包

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据(版本号110包序号00包总数64)	数据CRC16
主控板	23	01 00	33	01	07 00	54 A7	01 01 00 00 00 23 45	68 AC
平台	23	01 00	33	01	31 10	42 CB	01 01 00 00 00 23 45 (单包数据)	

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据(版本号110包序号00包总数64)	数据CRC16
主控板	23	01 00	33	01	07 00	54 A7	01 01 00 01 00 23 45	D3 2E
平台	23	01 00	33	01	31 10	42 CB	01 01 00 01 00 23 45 (单包数据)	

.....

设备收到升级通知后，通过升级审核，发送第一包数据请求（包序号为0开始）

然后再请求第二包.....

数据结构解析：

	数据类型	长度	范围
软件版本号	uint8_t * 3	3	
包序号	uint16_t	2	1-65535
包大小	uint16_t	2	1-65535
数据包总数	uint16_t	2	
数据分包		<= 1 KB	

注意：包序号从0开始。

52号命令：升级完成确认

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	52	0	5	软件版本号+数据包总数
设备	设备地址	52	10	0	无

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据(版本号110包总数64)	数据CRC16
平台	23	01 00	34	00	05 00	05 73	01 01 00 40 00	29 FC
主控板	23	01 00	34	0A	00 00	26 21		

上位机给设备下发，表示升级完成，设备收到命令后重启，由bootloader完成固件升级。

53号命令：升级取消

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	53	0	2	数据包总数
设备	设备地址	53	10	0	无

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据(包总数64)	数据CRC16
平台	23	01 00	35	00	02 00	06 BF	40 00	30 70
主控板	23	01 00	35	0A	00 00	06 BF	27 DD	

4 日志协议

54号命令：日志编号和内容获取

日志编号获取

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	54	0	1	日志类型
设备	设备地址	54	0	9	日志编号

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据（日志类型 首日志编号 尾日志编号）	CRC16
平台	23	01 00	36	00	01 00	06 0B	01	7E 80
主控板	23	01 00	36	00	09 00	01 CB	01 00 00 00 00 02 00 00 00	66 3B

日志类型：如黑盒日志（周期性记录）、状态日志等。

类型	值
黑盒日志	0
状态日志	1

日志编号数据结构如下：

```
/* 日志编号结构体 */
typedef __packed struct {
    uint8_t type; /* 日志类型 */
    uint32_t first; /* 第一个日志编号：从0开始，不循环 */
    uint32_t last; /* 最后一个日志编号：从0开始，不循环 */
} record_num_t;
```

日志内容获取

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	54	2	9	日志请求信息
设备	设备地址	54	2	1+日志信息*数量	日志类型+日志信息

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据	CRC16
平台	23	01 00	36	02	09 00	A0 0B	01 00 00 00 00 01 00 00 00	B5 D8
主控板	23	01 00	36	02	09 00	A0 0B	B5 D8	

日志类型：如黑盒日志（周期性记录）、状态日志等。

日志请求信息数据结构如下：

```

/* 日志请求信息结构体 */
typedef __packed struct {
    uint8_t type; /* 日志类型 */
    uint32_t first; /* 第一个日志编号：从0开始，不循环 */
    uint32_t count; /* 获取日志数量 */
} record_req_t;

```

日志信息数据结构如下：

```

/* 黑盒日志结构体（和黑盒标题项对应） */
typedef __packed struct {
    rtc_time_t time; /* RTC时间 */
    uint16_t dev_state; /* 设备状态 */
    uint8_t rsvd[2]; /* 填充值 */
} bbox_record_t;

/* 状态日志结构体（和状态标题项对应） */
typedef __packed struct {
    uint8_t state;
    uint8_t rsvd[3];
    uint32_t time_sec; /* 状态时间：从1970-01-01 00:00:00起的秒数 */
} state_record_t;

typedef __packed struct { //8字节
    uint16_t year; /* 年：2000-2099 */
    uint8_t month; /* 月：1-12 */
    uint8_t mday; /* 日：1-31 */
    uint8_t wday; /* 星期：1-7 */
    uint8_t hour; /* 时：0-23 */
    uint8_t minute; /* 分：0-59 */
    uint8_t second; /* 秒：0-59 */
} rtc_time_t;

```

55号命令： 日志停止状态查询和设置

停止状态查询

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	55	0	1	日志类型
设备	设备地址	55	0	4	停止状态

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据	CRC16
平台	23	01 00	37	00	01 00	07 F7	01	7E 80
主控板	23	01 00	37	00	04 00	04 A7	FF FF FF FF	01 B0

停止状态：如没有停止记录、停止持续时间、永久停止等。

类型	值
没有停止记录	0
停止持续时间	单位为秒
永久停止	~0（0xFFFFFFFF）

停止状态设置

时序	地址	命令号	操作类型	数据长度	数据
平台	设备地址	55	1	9	停止状态
设备	设备地址	55	1	1	日志类型

示例：

源头	#	地址	命令号	操作类型	数据长度	包头CRC16	数据（1000秒）	CRC16
平台	23	01 00	37	01	09 00	A0 0B	01 64 00 00 00	06 F0
主控板	23	01 00	37	01	01 00	A0 0B	01	7E 80

停止状态数据结构如下：

```
/* 记录停止信息 */
typedef __packed struct {
    uint8_t type;      /* 日志类型 */
    uint32_t start;     /* 开始时间戳（从1970-1-1 00:00:00起的秒数） */
    uint32_t duration; /* 持续时间（秒），0=不停止，~0=永久停止 */
} record_stop_t;
```

附录

CRC16校验方法C语言实现如下：

```
/*
 * 功能描述：此函数计算字节流的CRC16（ModBus RTU，0xA001）
 * 入参说明：buff --- 数据缓冲区
 *             len --- 数据长度（字节）
 * 返回值：CRC校验结果
 */
unsigned short crc16_RTU(const void *buff, int len) {
    const unsigned char *data = (const unsigned char *)buff;
    unsigned short      crc   = 0xffff;
    unsigned int         idx;

    while (len-- != 0) {
        idx = (crc & 0x0f) ^ (*data & 0x0f);
        crc = crc >> 4;
        crc ^= CRC16_RTU_TABLE[idx];
        idx = (crc & 0x0f) ^ ((*data >> 4) & 0x0f);
        crc = crc >> 4;
        crc ^= CRC16_RTU_TABLE[idx];
        data++;
    }

    return crc;
}
```

协议更新记录

版本号	时间	作者	更新记录
V1.0	2024/11/27	陈军	初始版本
V1.1	2024/12/11	陈军	1) 增加日志协议 2) 更新固件升级协议